scCO■ Automated Pressure Regulation System

Interview Preparation Guide

Vijai Venkatesh • University of Michigan • 2026

1. Project Overview & Resume Bullet

This project is a **closed-loop pressure control system** for a supercritical CO■ (scCO■) research vessel. A Raspberry Pi runs a Python PID controller that autonomously pressurizes a sealed PARR pressure vessel with CO■ to a configurable target (up to **28 MPa / 4061 PSI**), holds it stable within ±5 kPa, then safely depressurizes — all without human intervention.

CO■ becomes supercritical above **31.1°C / 7.38 MPa**. The system operates at **60°C**, so CO■ crosses the supercritical threshold during the ramp and stays supercritical for the entire hold phase.

Resume Bullet (full version)

Built a closed-loop scCO■ pressure regulation system on Raspberry Pi: adaptive PID controller with Peng-Robinson EOS-based gain scheduling, Tkinter live dashboard, CSV logging, and ThingSpeak cloud upload; full physics simulation runs on desktop for development without hardware.

Resume Bullet (1-line version)

Built automated scCO■ pressure control system (Raspberry Pi, Python PID, 28 MPa) with real-gas physics model, adaptive gain scheduling, and live GUI dashboard.

What Makes This Project Stand Out

- Real physics, not approximations — Peng-Robinson EOS, not ideal gas law
- Adaptive control — gains change every 100 ms as the plant changes character
- Full simulation stack — desktop SIL before any hardware exists
- Safety-first design — hardware E-stop, software watchdog, overpressure trip, rate limiter
- Multithreaded architecture — control (10 Hz), GUI, and cloud on separate threads
- Clean HAL design — one-line swap between SimulatedHardware and RPiHardware

2. System Architecture

Thread Architecture

Thread	Rate	Responsibility
Control Loop	10 Hz (100 ms)	State machine, PID, hardware I/O, CSV logging
GUI Thread	10 Hz refresh	Tkinter mainloop, chart, button events
Cloud Thread	Every 2 s	ThingSpeak upload (daemon, non-blocking)
Physics Sim Thread	10 Hz	Simulated hardware — PR-EOS pressure/temp dynamics

Hardware Abstraction Layer (HAL)

Both **SimulatedHardware** and **RPiHardware** inherit from **HardwareBase** (abstract). To switch between simulation and real hardware, change ONE line in config.py:

```
SIMULATION = True   # desktop (Windows/macOS)
SIMULATION = False  # Raspberry Pi, real sensors
```

The control loop, state machine, and GUI never know which backend is running.

Key Files

File	Purpose
main.py	Entry point; wires all threads together
config.py	ALL constants — edit this first for any tuning
gui.py	Tkinter + Matplotlib live dashboard
hardware/base.py	Abstract HAL interface (never edit)
hardware/simulator.py	PR-EOS physics simulation at 10 Hz
hardware/rpi_hardware.py	RPi stub — needs GPIO/ADS1115 wiring (next step)
control/state_machine.py	IDLE → PRESSURIZE → HOLD → DEPRESSURIZE
control/pid.py	Discrete PID: anti-windup + derivative-on-measurement
control/gain_scheduler.py	8-point adaptive gain table vs pressure
control/ramp.py	Linear setpoint ramp generator
control/eos.py	Peng-Robinson EOS for CO■ — core physics
logger.py	CSV data logger (every 100 ms)
cloud.py	ThingSpeak background uploader

3. Control Theory — PID & Gain Scheduling

PID Controller (control/pid.py)

Discrete-time PID at 10 Hz. Two key features beyond a basic PID:

- **Anti-windup**: integral accumulates only when output is NOT saturated. Prevents the integrator from winding up during a long ramp where the valve is already fully open.
- **Derivative on measurement** (not on error): D-term is –Kd×Δmeasurement/dt. This eliminates the derivative spike caused by sudden setpoint steps.

```
# Anti-windup: only integrate when not saturated
if output_min < last_output < output_max:
    integral += error * sample_time

# Derivative on measurement (no setpoint kick)
d_term = -kd * (measurement - prev_measurement) / sample_time
```

Gain Schedule — 8-Point Table

Pressure (MPa)	Kp	Ki	Kd	Phase / Physics Reason
0.0	50	6.0	2.5	Low pressure — compressible gas, moderate ΔP
5.0	58	7.0	2.5	Rising effort to maintain ramp rate
10.0	65	8.0	2.0	Peak Kp — max compressibility, most push needed
17.2	60	7.0	1.5	← 2500 PSI crossover: CO■ stiffens (PR-EOS inflection)
20.0	45	5.0	1.0	Dense phase — back off, hair-trigger response
24.0	32	4.0	0.8	Approaching target — smooth fine-approach
27.0	22	3.0	0.5	Near target — minimal actuation
28.0	18	3.0	0.5	At target — switch to HOLD gains (Kp=35, Ki=12, Kd=0.5)

Is the Gain Schedule Physics-Backed?

Shape: yes. Exact numbers: empirically refined from physics starting points.

- **Why gains rise 0→10 MPa**: Flow model is dP = K×valve×√(ΔP). As vessel pressure rises, ΔP shrinks → less flow per valve %. Plant gain drops → Kp must rise. Classic: Kp_controller ∝ 1/plant_gain.
- **Why gains fall sharply after 17.2 MPa**: PR-EOS compressibility Z collapses at the supercritical transition. At 10 MPa Z≈0.72; at 20 MPa Z≈0.46 (54% denser than ideal gas). A tiny valve move causes a huge pressure spike.
- **17.2 MPa crossover**: inflection in dZ/dP from PR-EOS; confirmed in lab.
- **Gas factors** (CO■=1.0, N■=0.80, Ar=0.85): molar mass & density ratios — pure physics.
- **Exact numbers** (50, 65, 18…): seeded from physics, refined in Simulink PID Tuner.

"The shape of the schedule — rising at low pressure, peak at 10 MPa, steep fall past 17 MPa — is predicted by the flow equation and PR-EOS compressibility. The specific numbers were seeded from physics and refined using Simulink PID Tuner."

How Over-Compensation Is Prevented

- **Derivative-on-measurement**: damps rapid pressure rises before they overshoot
- **Gain scheduler**: reduces Kp aggressively from 65→18 as pressure nears target
- **Ramp setpoint generator**: setpoint never jumps — ramps at 350 kPa/min, so tracking error is always small and controlled

4. Peng-Robinson EOS — Why Real Gas Physics Matter

Why Not Ideal Gas?

Ideal gas: PV = nRT. At 20 MPa / 60°C, CO■ has compressibility factor **Z = 0.46** (actual volume is 54% less than ideal predicts). Using ideal gas law gives wrong flow rates, wrong temperature-pressure coupling, and incorrect PID tuning.

Peng-Robinson EOS

```
P = RT/(V-b) - a(T)/[V(V+b) + b(V-b)]
```

```
CO2 constants:
  Tc = 304.13 K,  Pc = 7.3773 MPa,  omega = 0.22394
  R  = 8.314e-6 MPa*m3/mol*K
  b  = 2.664e-5  m3/mol
```

Key Function: dP_from_dT

Used every physics step to compute pressure change from temperature fluctuation:

```
def dP_from_dT(P_MPa, T_K, dT_K):
    # Solve PR-EOS for molar volume V at (P, T)
    # Compute (dP/dT)_V = R/(V-b) - (da/dT)/[V(V+b)+b(V-b)]
    return dP_dT * dT_K
```

Why This Matters Practically

Condition	Ideal Gas Predicts	PR-EOS Reality
0.01°C temp flicker at 20 MPa	~0.6 kPa pressure change	3.5–4.7 kPa change (5.9× more!)
Density at 20 MPa / 60°C	Z = 1.0 (baseline)	Z = 0.46 (118% denser)
HOLD gain needed	Low Kp sufficient	Must use tight HOLD Kp=35 (2× ramp value)

Interview point: "A 0.01°C INKBIRD temperature flicker causes 4 kPa of pressure noise at 20 MPa — 5.9× what ideal gas predicts. Without PR-EOS, HOLD accuracy would be terrible."

5. State Machine & Safety Systems

State Machine (control/state_machine.py)

State	Entry Condition	Action	Exit Condition
IDLE	Startup / E-stop / depressurize complete	Valve=0%, solenoid=0%	START button pressed
PRESSURIZE	START pressed & temp ≥55°C	Ramp setpoint 350 kPa/min, PID drives valve	Pressure within 0.05 MPa of target
HOLD	Target pressure reached	Tight PID (HOLD gains), valve & solenoid active	DEPRESSURIZE button or E-stop
DEPRESSURIZE	User command	Ramp setpoint down at 2 MPa/min, solenoid vents	Pressure ≤0.2 MPa → IDLE

Safety Layers (priority order)

- **Hardware E-stop button**: NC contact on GPIO 23, cuts power immediately
- **Software E-stop (GUI)**: calls sm.request_estop() → valve=0, solenoid=0, IDLE
- **Overpressure software trip**: pressure >28.5 MPa → emergency stop
- **Rate limiter (WARNING-only)**: logs at >600 kPa/min; forces IDLE only at >5000 kPa/min (catastrophic). No bang-bang — prevents chattering.
- **Temperature gate**: START locked until temp ≥55°C
- **Watchdog**: WATCHDOG_TIMEOUT_S; misses → emergency stop
- **MAWP margin**: 28 MPa operating vs 34.47 MPa PARR MAWP = 18.7% safety margin

Rate Estimation — 5-Second Rolling Window

Single-sample dP/dt at 10 Hz with 5 kPa noise → ~3000 kPa/min apparent rate (false alarm). 5-second window reduces this well below 600 kPa/min threshold. Requires ≥3 seconds of data before trusting the estimate.

```
if window_dt < 3.0:
    return last_smoothed_rate   # not enough data
rate = abs((p_now - p_5s_ago) / window_dt * 60.0)  # MPa/min
```

6. Hardware Inventory

Component	Make / Model	Key Spec
Pressure Vessel	PARR 2302HC, 316 SS	1 L, MAWP 34.47 MPa (5000 PSI) @ 350°C
Gas Booster	HII 5G-TD-28/150-CO2	Max outlet 172 MPa; modeled as 50 MPa supply
Pressure Transducer	Ashcroft 0–5000 PSI, 4–20 mA	ADS1115 A0, 250Ω shunt → 1–5 V
Temp Controller	INKBIRD, 60°C setpoint	±0.01°C settled; ±0.3°C during heat-up
ADC	ADS1115, 16-bit I²C	Channels A0 (pressure), A1 (temp)
Stepper Driver	A4988, 1/16 microstepping	GPIO 17 STEP, GPIO 27 DIR, GPIO 22 EN
Stepper Motor	NEMA 17, 200 steps/rev	Actuates motorized needle valve
Solenoid Valve	24VDC relay-driven vent	GPIO 18 PWM (hardware channel 0)
E-Stop	NC push button	GPIO 23 BCM, pulled HIGH, LOW when pressed
Controller	Raspberry Pi 4 (4 GB)	Python 3.11, 10 Hz control loop
Power Supply	Dayton 33NT20, 24VDC 50W	DIN rail mount

Gas Inventory

- CO■ (Bone Dry, UN1013) — primary process gas, Metro Welding Supply Detroit
- N■ (Ultra High Purity, UN1066) — alternate experiment gas
- Ar (Prepurified, UN1006) — alternate experiment gas
- Air (Dry Grade, UN1002) — drives booster pump only, never enters vessel

7. Simulation → Hardware Roadmap

Current Status: Software-in-the-Loop (SIL)

Complete control algorithm runs in simulation on a desktop PC. SimulatedHardware replaces all physical sensors and actuators with PR-EOS physics. The same code runs on RPi with one config change.

Stage	Description	What's Real
SIL (now)	Desktop Python simulation	Algorithm only — physics is mathematical
HIL (next)	RPi runs control code vs real hardware	RPi + real sensors + real actuators
Fully Real	RPi in control loop, PARR vessel pressurized	Everything — all hardware live

Completing rpi_hardware.py (Blocking Step 1)

- File exists with all code commented out — just uncomment and fix imports
- pip install adafruit-circuitpython-ads1x15 adafruit-blinka RPi.GPIO
- ADS1115 A0: pressure transducer 4–20 mA via 250Ω → 1–5 V → 0–34.47 MPa
- ADS1115 A1: thermocouple amplifier 0–5 V → 0–100°C
- NEMA 17 stepper: STEP/DIR/EN on GPIO 17/27/22
- Solenoid: hardware PWM on GPIO 18

Simulink Digital Twin (simulink/ folder)

- build_model.m — creates scCO2_plant.slx using Simscape Fluids
- run_simulation.m — runs sim, plots step response
- export_gains.m — exports Kp/Ki/Kd from MATLAB PID Tuner
- Status: not yet tested (needs MATLAB + Simscape Fluids toolbox)

PID Tuning Plan (HOLD gains need re-tuning)

- Run simulation to 20 MPa, watch HOLD behavior with PR-EOS noise
- Target: hold ±5 kPa at 28 MPa with 0.01°C INKBIRD noise input
- Ziegler-Nichols: ramp Kp until sustained oscillation → extract Ku, Pu
- MATLAB PID Tuner: linearized plant model → automatic tuning → export gains

How This Maps to GNC / Embedded Systems Work

"It's a complete real-time control system: sensors → state estimator (noise-rejected rate) → state machine → guidance law (ramp setpoint) → feedback controller (PID) → actuators, with safety watchdogs and a hardware abstraction layer. The SIL → HIL → real-hardware progression mirrors how aerospace GNC systems are developed. The adaptive gain scheduling based on a physics model of the plant is the same design principle used in gain-scheduled flight control laws for varying dynamic pressure."

8. Key Interview Q&A

Q: Describe the project in 30 seconds.

"I built a closed-loop pressure control system for a scCO■ research vessel. A Raspberry Pi runs a Python PID controller that ramps the vessel from ambient to 28 MPa at a controlled rate, holds it stable within ±5 kPa, then safely vents. The control uses adaptive gain scheduling based on a Peng-Robinson EOS real-gas model. I have a full physics simulation that runs on desktop so I can develop and tune without hardware."

Q: Why adaptive gain scheduling instead of fixed PID?

"The plant changes character dramatically with pressure. At low pressure CO■ is highly compressible — the valve has low authority. Near 28 MPa CO■ is dense and near-incompressible — the same valve opening causes a huge overshoot. A fixed Kp good for low pressure would be unstable near target; one good for near target would be sluggish at low pressure. Gain scheduling is the right tool when plant gain varies significantly across the operating range."

Q: How does this relate to Hardware-in-the-Loop testing?

"Currently we're in Software-in-the-Loop (SIL) — the control algorithm runs against a mathematical physics simulation. The next step is HIL: real Raspberry Pi and real sensors/actuators, initially at a safe low pressure. SIL proves the algorithm before we run it near real hardware at 4000 PSI. The HAL design (one-line swap sim → RPi) is what makes this progression clean."

Q: How did you handle sensor noise in the rate estimator?

"Single-sample dP/dt at 10 Hz with 5 kPa transducer noise gives an apparent rate of ~3000 kPa/min even when the vessel is static — that would constantly trigger the rate limiter. I use a 5-second rolling window, which reduces noise by the square root of the window length. I also require ≥3 seconds of data before trusting the estimate."

Q: Why derivative-on-measurement instead of derivative-on-error?

"When the setpoint steps, the derivative of the error has an instantaneous spike ('derivative kick'), feeding a huge D-term into the output for one sample. Since the setpoint ramps slowly (350 kPa/min), derivative-on-measurement and derivative-on-error are nearly identical most of the time, but measurement avoids the kick on any sudden setpoint change."

Q: What would break first if you deployed today?

"rpi_hardware.py — it's a stub with raise NotImplementedError everywhere. The GPIO wiring for ADS1115, stepper driver, and solenoid PWM needs to be uncommented and tested. That's the immediate blocking task before any hardware deployment. After that, HOLD PID gains need re-tuning against real sensor noise."

Q: How does the project map to GNC / embedded systems work?

“It’s a complete real-time control system: sensors → state estimator → state machine → guidance law → feedback controller → actuators, with safety watchdogs and a hardware abstraction layer. The SIL → HIL progression mirrors aerospace GNC development. Adaptive gain scheduling based on a physics model of the plant is the same principle used in gain-scheduled flight control laws for varying dynamic pressure.”

9. Complete Parts List

High-Pressure Fluid System (Swagelok / Parker)

Item	Brand / Model	Spec	Est. Cost
Motorized needle valve body	Parker NP6: 4F-NP6LK-SSP	SS316, 1/4" tube, 60,000 PSI rated	~$400
Motor coupling	NEMA 17 stepper + custom bracket	200 steps/rev, A4988 1/16 step	~$30
Check valve	Swagelok SS-CHS4-5	1/4" tube, 6000 PSI (not SS-4C — 3000 PSI only)	~$150
Manual ball valve ×3	Swagelok SS-83KS4	1/4" tube, 6000 PSI (not SS-43GS4 — 3000 PSI only)	~$450
Compression fittings	Swagelok SS-400 series	1/4" OD, 316 SS (assorted)	~$200
SS tubing	316 SS, 1/4" OD × 0.065" wall	High-pressure rated, 10 ft	~$80
Relief valve	Parker 442F42	SS316, 1/4" MNPT	~$165

Electronics & Control (Grainger / Amazon)

Item	Brand / Model	Est. Cost
Pressure transducer	Ashcroft 0–5000 PSI 4–20 mA SS IP67 (Grainger K4708)	~$400
Pressure gauge (mechanical)	Ashcroft 0–5000 PSI (Grainger K4201)	~$100
DIN rail PSU 24VDC 50W	Dayton 33NT20 (Grainger)	~$55
DIN rail relay 24VDC	Generic DIN relay 24VDC coil	~$20
Raspberry Pi 4 (4 GB)	Raspberry Pi Foundation	~$55
ADS1115 ADC module	Adafruit breakout	~$10
NEMA 17 stepper motor	Generic 200 steps/rev	~$15
A4988 stepper driver	Pololu A4988	~$8
32 GB microSD	SanDisk	~$10
Relay module 24VDC	Generic	~$10
PTFE tape	Grainger 34P209	~$5

Optional: Windowed Vessel (for scCO■ flow visualization)

- Autoclave Engineers (Parker): EZE-Seal series, sapphire windows, up to 60 MPa
- Sitec Reactor Technology: 100–600 mL, sapphire/borosilicate windows, up to 100 MPa
- HiP (High Pressure Equipment Co.): modular sight windows, SS316, 10,000 PSI
- Typical cost: $2,000–$8,000 depending on volume and window spec

Estimated Total Cost

Category	Estimated Total

High-pressure fluid system	~$1,325
Electronics & control	~$678
Vessel (PARR 2302HC — already owned)	$0
Gas cylinders (already owned)	$0
TOTAL	~$2,003